

WebSocket & Redis

A Clear, Step-by-Step Guide for Real-Time Websites

This guide explains what WebSocket is, what Redis is, why each one is used, and exactly how they work together in a real-time website like a chat application. Every section builds on the previous one.

Part 1 — WebSocket

1.1 What is WebSocket?

WebSocket is a way for a website (the browser) and a server to keep **one connection open** and send data to each other in real time.

```
Browser  <----->  Server
          WebSocket
          connection
```

1.2 Why do we use WebSocket?

We use WebSocket when the server needs to send something to the browser **immediately**, without the browser having to repeatedly ask "do I have anything new?"

Example: in a chat app, when User A sends a message, User B should see it instantly:

```
User A
  |
  v
Server
  |
  v
User B
```

The key difference from normal HTTP:

Normal HTTP	WebSocket
Browser sends a request, server sends one response, then the connection ends.	Browser and server open one connection and keep it open. Either side can send data at any time.

1.3 How WebSocket works — 3 stages

Every WebSocket connection goes through three basic stages:

1. **Connect** — the browser opens a connection to the server.
2. **Send / Receive** — both sides can send data while the connection is open.
3. **Disconnect** — the connection closes when the user leaves.

Step 1 — Connect

When a user opens your website, the browser creates a WebSocket connection to your server.

```
Browser
  |
```

```
| Connect  
v
```

Server

If accepted:

```
Browser  <=====>  Server  
          OPEN
```

This connection now stays open.

Step 2 — Send data

Suppose User A types "Hello". The browser sends it through the open WebSocket connection:

User A

```
|
```

```
v
```

Browser

```
|
```

```
v
```

WebSocket

```
|
```

```
v
```

Server (receives "Hello")

Step 3 — Server sends data back

The server can also push data to the browser at any time (this is the whole point of WebSocket):

Server

```
|
```

```
v
```

WebSocket

```
|
```

```
v
```

Browser (e.g. "Message received")

So WebSocket works in **both directions** — Browser to Server, and Server to Browser — over the same open connection.

1.4 Simple chat example

Two users connect to the same server:

```
User A           Server           User B  
|---- Connect ----->|           |  
|                       |<---- Connect -----|
```

Now both are connected:

User A <----- Server -----> User B

When A sends "Hello", the flow is:

A -> WebSocket -> Server -> WebSocket -> B

The server is the one responsible for deciding: "this message from A is meant for B" — and then forwarding it over B's open connection.

1.5 What happens when a user leaves?

If User A closes the website, the browser disconnects and the server closes that connection:

```
A
|
v
Disconnect
|
v
Server (connection closed)
```

1.6 Complete WebSocket lifecycle

```
USER OPENS WEBSITE
|
Browser creates WebSocket connection
|
Server
|
Connection established
|
Connection stays open
|           |
Browser->Server  Server->Browser
  Send         Receive
|_____|
|
Connection closes
|
Disconnect
```

In one sentence: WebSocket creates a persistent, two-way connection between browser and server so both can send data to each other instantly, without repeated requests.

Part 2 — Redis

2.1 What is Redis?

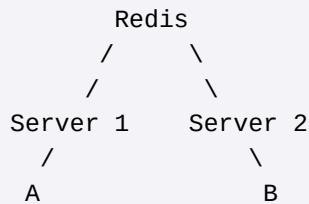
Redis is a very fast, in-memory data store. It is used for **temporarily storing data** and **sharing information between servers**.

```
Server <-----> Redis
```

Redis primarily keeps its data in memory (RAM) instead of on disk, which is why reading and writing to it is extremely fast compared to a normal database.

2.2 Why do we need Redis if we already have WebSocket?

A single WebSocket server can connect User A and User B directly — no Redis needed. But real websites usually run **multiple servers** for scale and reliability. The problem: each server only knows about its own connections.



If A is connected to Server 1 and B is connected to Server 2, Server 1 physically cannot send a message directly to B's browser — it doesn't hold that connection. Redis becomes the shared middle point that lets the two servers talk to each other.

```
A
|
v
Server 1
|
v
Redis
|
v
Server 2
|
v
B
```

2.3 What exactly does Redis store?

In a WebSocket + chat system, Redis is typically used to hold:

Connection info — which user is connected to which server:

```
User A -> Server 1 -> Connection 101
User B -> Server 2 -> Connection 205
```

Pending messages — messages waiting for an offline user:

```
User B -> "Hello"
User B -> "How are you?"
```

Temporary state — short-lived info like "typing..." or online status:

```
User A is typing
User B is online
User C is in room 123
```

2.4 Offline users — how Redis helps

If B is completely offline when A sends a message, the server stores it temporarily in Redis:

```
A sends message
  |
  Server
  |
Is B online? ---- NO
  |
  Redis
  |
B's pending message stored
```

When B later comes online, the server checks Redis and delivers what was waiting:

```
B opens website
  |
  WebSocket
  |
  Server ---- checks ----> Redis
  |                               |
  |<----- "Hello" -----|
  |
  WebSocket
  |
  B (sees the message)
```

2.5 Redis is NOT WebSocket

This is an important distinction. The browser **never** talks to Redis directly. Only the server talks to Redis. The browser only ever talks to the server, over WebSocket.

```
Browser <--WebSocket--> Server <--Redis connection--> Redis
```

2.6 One user, multiple devices

Redis (together with the server) can track that one user has several active connections at once:

```
User B
|-- Phone -> Connection 101
|-- Laptop -> Connection 102
```

When A sends B a message, your application decides whether to deliver it to the phone only, the laptop only, or both — this is an application-level choice, not something WebSocket or Redis decides automatically.

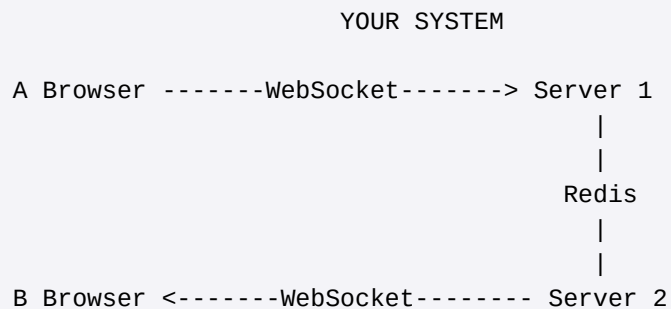
2.7 A warning about Redis

Redis is fast, but it is **temporary** storage — data in Redis can be lost or expired. Don't use Redis as a replacement for a real database. The typical, safer design is:

```
WebSocket -> real-time delivery
Redis      -> fast temporary data / server-to-server coordination
Database   -> permanent message history
```

Part 3 — WebSocket + Redis Together

3.1 The complete picture



WebSocket connects users to your server for real-time, two-way communication. **Redis** helps your servers share information with each other and can temporarily hold data such as pending messages or connection info.

3.2 Full message flow, all cases

Case A — B is online, same server as A

A -> WebSocket -> Server -> WebSocket -> B

Case B — B is online, on a different server

A -> WebSocket -> Server 1 -> Redis -> Server 2 -> WebSocket -> B

Case C — B is offline

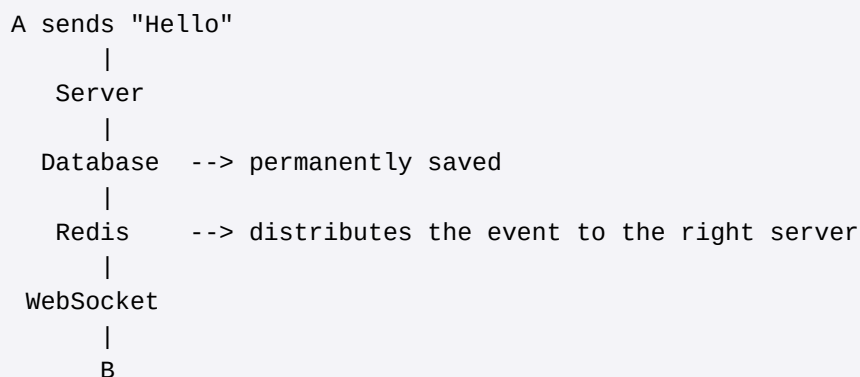
A -> WebSocket -> Server -> Redis -> (message waits)

... B comes online later ...

B -> WebSocket -> Server -> Redis -> delivers stored message -> B

3.3 With a permanent database added

In a production chat app, you'd typically save every message to a database as well, so history is never lost even if Redis data expires:



3.4 The four lines to remember

If you remember nothing else, remember this:

1. Browser -> WebSocket -> Server
2. Server -> Redis -> Server (only when servers need to talk to each other)
3. Server -> WebSocket -> Browser
4. Database -> permanently stores important data

Summary: WebSocket = communication (keeps a live connection open between browser and server).
Redis = fast shared storage (lets multiple servers coordinate and temporarily holds data like pending messages). Together, they let a chat-style website deliver messages instantly, even across multiple servers and devices, while offline users still get their messages later.